

AI Usage Note – AIForge

Overview

This document outlines how Artificial Intelligence (AI), specifically Large Language Models (LLMs) and AI Agents, was utilized during the development and execution of the AIForge platform.

AIForge is an AI-powered Software Testing Lifecycle Platform designed to automate the complete testing workflow, including requirement analysis, test generation, automation script creation, test data generation, execution support, bug analysis, self-healing automation, repository intelligence, and CI/CD pipeline generation.

What AI Helped With

Requirement Analysis

AI was used to analyze user stories and software requirements to identify:

- Functional requirements
- Validation rules
- Acceptance criteria
- Testing objectives

This significantly reduced the effort required to manually interpret requirements before test design.

Intelligent Test Generation

AI generated comprehensive test suites covering:

- Positive Test Cases
- Negative Test Cases
- Edge Cases
- Security Test Cases

The generated tests provided broader coverage and improved consistency compared to manually created test cases.

Multi-LLM Decision Making

AIForge integrates multiple LLM providers:

- Gemini
- Llama (Groq)
- DeepSeek

A custom Multi-LLM Orchestrator was implemented to collect responses from multiple models, compare outputs, and select the most suitable result for downstream processing.

Automation Script Generation

AI assisted in generating automation scripts using:

- Python
- Java
- JavaScript
- Selenium
- Playwright
- JUnit

Generated outputs included executable test scripts, assertions, and framework-specific automation structures.

Test Data Generation

AI automatically generated:

- Valid Test Data
- Invalid Test Data
- Boundary Value Inputs
- Security Testing Inputs

This reduced the manual effort involved in creating large and diverse testing datasets.

Bug Analysis & Root Cause Detection

AI was used to analyze failed test execution logs and generate:

- Root Cause Analysis
- Failure Classification
- Recommended Fixes

This helped accelerate debugging and defect investigation activities.

Repository Intelligence

AI analyzed project repositories and application structures to identify:

- Frameworks
- APIs
- Modules
- Features

The extracted information was used to improve context-aware test and automation generation.

Documentation Assistance

AI assisted in generating:

- Test Documentation
- Cucumber (BDD) Documentation
- Reports
- Technical Documents

What AI Got Wrong (and How We Fixed It)

Duplicate and Incomplete Test Cases

Different LLMs occasionally produced duplicate or partially complete test scenarios.

Fix:

An AI Judge Agent was developed to validate outputs, remove duplicates, and improve test coverage before final delivery.

Generic Automation Scripts

Initial automation scripts sometimes lacked application-specific understanding.

Fix:

A Project Context Agent was implemented to analyze project structure and generate more accurate, context-aware automation scripts.

Hallucinated Framework Components

AI occasionally referenced non-existent classes, methods, or framework components.

Fix:

Generated outputs were validated through rule-based verification and execution-level checks before export.

Locator Instability

Automation scripts occasionally failed when UI elements changed.

Fix:

A Self-Healing Locator Agent was introduced to automatically identify and repair broken Selenium locators.

Human Contributions

The project team was responsible for:

- System Architecture Design
- Multi-Agent Workflow Design
- FastAPI Backend Development
- Agent Development and Integration
- Database Design
- API Development
- Workflow Orchestration
- Export Module Development
- Testing and Validation

AI was used as an enabling technology, while the architecture, implementation, integration, and quality assurance processes were designed and executed by the development team.

Key Prompts Used**Requirement Analysis Prompt**

"Analyze the following user story and identify functional requirements, acceptance criteria, validations, and testing objectives."

Test Generation Prompt

"Generate positive, negative, edge-case, and security test cases for the given requirement."

Automation Generation Prompt

"Generate executable automation scripts for the provided test cases using the selected framework."

Bug Analysis Prompt

"Analyze the execution logs and identify the root cause, impact, confidence score, and recommended fix."

Test Data Generation Prompt

"Generate valid, invalid, boundary, and security-focused test data for the provided scenarios."

Conclusion

AIForge demonstrates the practical application of Artificial Intelligence in software testing by combining multiple AI agents and LLMs to automate significant portions of the testing lifecycle. The platform enhances testing efficiency, improves coverage, reduces manual effort, and supports faster software delivery while maintaining human oversight throughout the development process.